

Project Title: Sports League & Event Ticketing Management System

Business Domain: Sports & Entertainment

1. Overview

The Sports League & Event Ticketing Management System manages sports league operations end to end, including team and player rosters, match scheduling, ticket inventory, match day operations, and fan loyalty programs. Built using an MVC architecture, it supports both Java (Spring MVC) and .NET (ASP.NET Core MVC) frameworks.

The system consists of five key modules:

1. Team & Player Roster Management
2. Match Schedule & Venue Management
3. Ticket Inventory & Booking
4. Match Day Operations & Score Tracking
5. Fan Loyalty & Merchandise Management

2. Assumptions

- **Database:** MySQL or SQL Server.
- **Roles:** League Administrator, Team Manager, Venue Operator, Ticketing Agent, Fan, Admin.
- **ORM:** Hibernate (Java), Entity Framework Core (.NET).
- Payment gateway and ticket scanning systems are simulated.
- Live score and statistics feeds use mock APIs.

3. Module-Level Design

3.1 Team & Player Roster Management Module

Purpose: Manages teams, players, and contract details.

Controller:

- TeamController
 - registerTeam()
 - addPlayer()
 - transferPlayer()

Q Search





- getTeamRoster()

Service:

- TeamService
 - Handles team registration, player contracts, and squad composition rules.

Model:

- Player Entity
 - Attributes:
 - playerId (PK)
 - teamId
 - playerName
 - position
 - jerseyNumber
 - contractStartDate
 - playerStatus (ACTIVE, INJURED, SUSPENDED, RETIRED)

3.2 Match Schedule & Venue Management Module

Purpose: Schedules league matches and manages venue allocations.

Controller:

- MatchController
 - scheduleMatch()
 - assignVenue()
 - rescheduleMatch()
 - getFixturesList()

Service:

- MatchService
 - Validates team availability, venue conflicts, and fixture rules.

Model:

- Match Entity
 - Attributes:



Search



- matchId (PK)
- homeTeamId
- awayTeamId
- venueName
- matchDate
- kickOffTime
- matchStatus (SCHEDULED, LIVE, COMPLETED, POSTPONED, CANCELLED)

3.3 Ticket Inventory & Booking Module

Purpose: Manages ticket inventory and customer bookings.

Controller:

- TicketController
 - createTicketCategory()
 - bookTicket()
 - cancelBooking()
 - getAvailableSeats()

Service:

- TicketService
 - Manages seat maps, dynamic pricing, and booking confirmations.

Model:

- TicketBooking Entity
 - Attributes:
 - bookingId (PK)
 - matchId (FK)
 - fanName
 - ticketCategory (GENERAL, PREMIUM, VIP, BOX)
 - seatNumber
 - ticketPrice
 - bookingStatus (CONFIRMED, CANCELLED, USED, REFUNDED)



3.4 Match Day Operations & Score Tracking Module

Purpose: Manages match day operations and records live scores.

Controller:

- MatchDayController
 - scanTicket()
 - recordScore()
 - logMatchEvent()
 - publishLiveCommentary()

Service:

- MatchDayService
 - Validates ticket entry, captures goals, fouls, and match incidents.

Model:

- MatchEvent Entity
 - Attributes:
 - eventId (PK)
 - matchId (FK)
 - eventMinute
 - eventType (GOAL, FOUL, CARD, SUBSTITUTION)
 - playerId
 - eventDescription

3.5 Fan Loyalty & Merchandise Management Module

Purpose: Manages fan loyalty programs and merchandise sales.

Controller:

- FanLoyaltyController
 - enrollFan()
 - creditLoyaltyPoints()
 - purchaseMerchandise()
 - redeemReward()



Search



**Service:**

- FanLoyaltyService
 - Calculates loyalty points, manages tier progression, and merchandise orders.

Model:

- FanProfile Entity
 - Attributes:
 - fanId (PK)
 - fanName
 - favoriteTeamId
 - loyaltyPoints
 - membershipTier (BRONZE, SILVER, GOLD)
 - memberSince

4. Database Schema**Player Table**

```
CREATE TABLE Player (  
  playerId INT AUTO_INCREMENT PRIMARY KEY,  
  teamId INT,  
  playerName VARCHAR(100),  
  position VARCHAR(50),  
  jerseyNumber INT,  
  contractStartDate DATE,  
  playerStatus ENUM('ACTIVE', 'INJURED', 'SUSPENDED', 'RETIRED')  
);
```

Match Table

```
CREATE TABLE Matches (  
  matchId INT AUTO_INCREMENT PRIMARY KEY,  
  homeTeamId INT,  
  awayTeamId INT,  
  venueName VARCHAR(100),  
  matchDate DATE,  
  kickoffTime TIME,  
  matchStatus ENUM('SCHEDULED', 'LIVE', 'COMPLETED', 'POSTPONED', 'CANCELLED')  
);
```

TicketBooking Table

```
CREATE TABLE TicketBooking (  
  bookingId INT AUTO_INCREMENT PRIMARY KEY,  
  matchId INT,  
  fanName VARCHAR(100),
```

```
ticketCategory ENUM('GENERAL', 'PREMIUM', 'VIP', 'BOX'),
seatNumber VARCHAR(10),
ticketPrice DECIMAL(10,2),
bookingStatus ENUM('CONFIRMED', 'CANCELLED', 'USED', 'REFUNDED'),
FOREIGN KEY (matchId) REFERENCES Matches(matchId)
);
```

MatchEvent Table

```
CREATE TABLE MatchEvent (
eventId INT AUTO_INCREMENT PRIMARY KEY,
matchId INT,
eventMinute INT,
eventType ENUM('GOAL', 'FOUL', 'CARD', 'SUBSTITUTION'),
playerId INT,
eventDescription VARCHAR(255),
FOREIGN KEY (matchId) REFERENCES Matches(matchId)
);
```

FanProfile Table

```
CREATE TABLE FanProfile (
fanId INT AUTO_INCREMENT PRIMARY KEY,
fanName VARCHAR(100),
favoriteTeamId INT,
loyaltyPoints INT,
membershipTier ENUM('BRONZE', 'SILVER', 'GOLD'),
memberSince DATE
);
```

5. Local Deployment Details

- Install DB + JDK 17/21/24 or .NET SDK 8.0
- Deploy using Tomcat or Kestrel

Steps:

1. Clone repository.
2. Run SQL schema.
3. Configure app settings files.
4. Build & run the project.

6. Conclusion

The Sports League & Event Ticketing Management System provides a robust MVC-based structure for league operations, ticketing workflows, and fan engagement. It is ideal for training scenarios and local deployment.